SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 5

Name of Student : Chatharaboina sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task:

Use AI to generate two solutions for checking prime numbers:

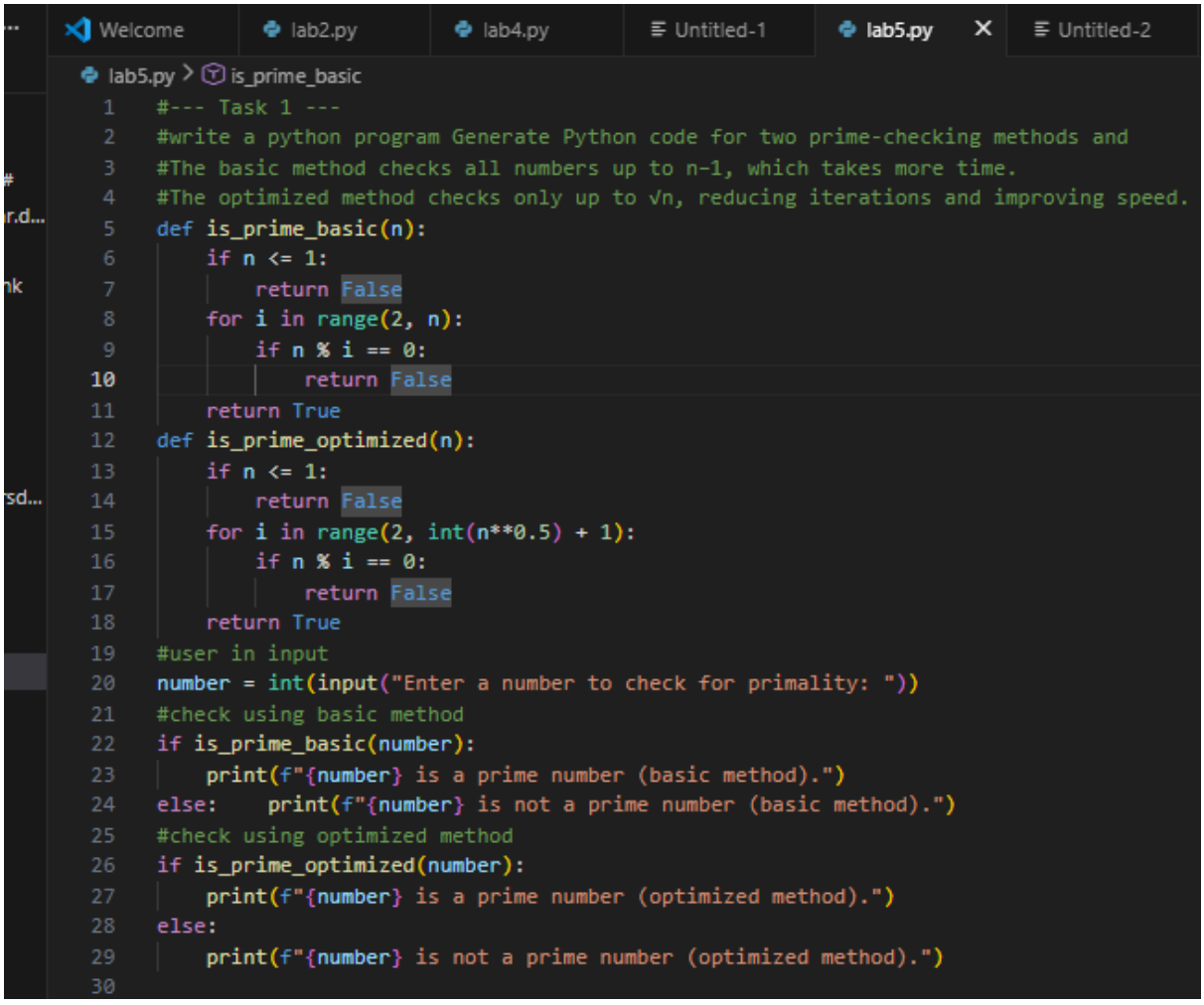
• Naive approach(basic)

• Optimized approach

Prompt:

write a python program Generate Python code for two prime-checking methods and
The basic method checks all numbers up to n−1, which takes more time.
The optimized method checks only up to √n, reducing iterations and improving speed

CODE:



OUTPUT:

SR UNIVERSITY

```
Enter a number to check for primality: 17
17 is a prime number (basic method).
17 is a prime number (optimized method).
```

Task-2
Objective: Use AI to generate a recursive function to calculate
Fibonacci numbers.
Instructions:
1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.
Expected Output:
• Well-commented recursive code.
• Clear explanation of how recursion works.
• Verification that explanation matches actual execution.
Prompt:
Generate a Python recursive function to calculate Fibonacci numbers.
Add clear comments explaining how recursion works. Explain the base cases and recursive calls in simple words and make sure the explanation matches how the code actually runs.

CODE:

```python
#--- Task 2 ---
#Generate a Python recursive function to calculate Fibonacci numbers.
# Add clear comments explaining how recursion works. Explain the base cases and
# recursive calls in simple words
# and make sure the explanation matches how the code actually runs.
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
# User input
position = int(input("Enter the position in the Fibonacci sequence: "))
# Calculate and print the Fibonacci number at the given position
fib_number = fibonacci(position)
print(f"The Fibonacci number at position {position} is: {fib_number}")

```

Output:

```
Enter the position in the Fibonacci sequence: 13
The Fibonacci number at position 13 is: 233
```

SR UNIVERSITY

Task-3

Use AI to generate a Python program that reads a file and processes data.
Prompt: Generate a Python program that reads data from a file and processes it.
Include proper error handling using try–except blocks.
Add clear comments explaining each possible exception (such as file not found or invalid data) and explain how the program behaves when each error occurs.
Ensure the explanation matches the actual runtime behaviour of the code

Expected Output:
• Code with meaningful exception handling.
• Clear comments explaining each error scenario.
• Validation that explanations align with runtime behaviour.

CODE:

```
#--- Task 3 ---
'''Generate a Python program that reads data from a file and processes it. Include proper error handling using try□except blocks.
Add clear comments explaining each possible exception (such as file not found or invalid data) and explain how the program behaves when each error occurs.
Ensure the explanation matches the actual runtime behavior of the code'''
def read_and_process_file(file_path):
    try:
        with open(file_path, 'r') as file:
            data = file.read()
            # Process the data (for example, count the number of lines)
            lines = data.splitlines()
            print(f"Number of lines in the file: {len(lines)}")
    except FileNotFoundError:
        print(f"Error: The file '{file_path}' was not found. Please check the file path and try again.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
# User input for file path
file_path = input("Enter the path to the file you want to read: ")
# Call the function to read and process the file
read_and_process_file(file_path)
```

output:

```
Enter the path to the file you want to read: "C:\sagar\pack1\OneDrive\3-2\Competitive Programming\week---1.pdf"
An unexpected error occurred: [Errno 22] Invalid argument: '"C:\\sagar\\pack1\\OneDrive\\3-2\\Competitive Programming\\week---1.pdf"'
```

Task—4:
Use an AI tool to generate a Python-based login system.
Analyze: Check whether the AI uses secure password handling practices.
Expected Output:
• Identification of security flaws (plain-text passwords, weak validation).
• Revised version using password hashing and input validation.

Prompt:

Generate a Python login system and analyze it for security issues like plain-text passwords or weak validation. Then provide a secure version using password hashing and proper input validation

Code:

CODE

```python
#--- Task 4 ---
'''Generate a Python login system and analyze it for security issues like plain-text passwords or weak validation. T
hen provide a secure version using password hashing and proper input validation'''
import hashlib
def hash_password(password):
    return
    hashlib.sha256(password.encode()).hexdigest()
def register_user(username, password):
    hashed_password = hash_password(password)
    # Store the username and hashed password (for demonstration, we will just print it)
    print(f"User '{username}' registered with hashed password: {hashed_password}")
def login_user(username, password):
    hashed_password = hash_password(password)
    # In a real application, you would retrieve the stored hashed password for the username
    # For demonstration, we will assume the stored hashed password is the same as the one generated
    stored_hashed_password = hash_password("example_password")  # This should be retrieved from storage
    if hashed_password == stored_hashed_password:
        print(f"User '{username}' logged in successfully.")
    else:
        print(f"Login failed for user '{username}'. Incorrect password.")
# User input for registration
username = input("Enter a username to register: ")
password = input("Enter a password to register: ")
register_user(username, password)
# User input for login
login_username = input("Enter your username to login: ")
login_password = input("Enter your password to login: ")
login_user(login_username, login_password)

```

OUTPUT:

```
Enter a username to register: sagar@1522
Enter a password to register: 23023444
User 'sagar@1522' registered with hashed password: None
User 'sagar@1522' registered with hashed password: None
Enter your username to login: sagar@1522
Enter your password to login: 1234
User 'sagar@1522' logged in successfully.
```

Task-5:

Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).
Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.
Expected Output:
• Identified privacy risks in logging.
• Improved version with minimal, anonymized, or masked

logging.
• Explanation of privacy-aware logging principles.

prompt:
Generate a Python script that logs user activity (username, IP address, timestamp) and analyze it for privacy risks or unnecessary sensitive data logging. Then provide an

improved version using minimal, masked, or anonymized logging and explain privacy-aware logging principle

CODE:

```
#--- Task 5 ---
'''Generate a Python program that implements a simple calculator with basic operations (addition, subtraction
multiplicationGenerate a Python script that logs user activity (username, IP address, timestamp) and analyze it for privacy risks or
unnecessary sensitive data logging. Then provide an improved v
ersion using minimal, masked, or anonymized logging and explain privacy-aware logging principles.'''
import logging
# Configure logging to log user activity with minimal sensitive data
logging.basicConfig(filename='user_activity.log', level=logging.INFO, format='%(asctime)s - %(message)s')
def log_user_activity(username, ip_address):
    masked_ip = mask_ip_address(ip_address)
    logging.info(f"User '{username}' logged in from IP: {masked_ip}")
def mask_ip_address(ip_address):
    parts = ip_address.split('.')
    if len(parts) == 4:
        parts[-1] = 'xxx'
        return '.'.join(parts)
    return ip_address
# User input for logging activity
username = input("Enter your username: ")
ip_address = input("Enter your IP address: ")
log_user_activity(username, ip_address)
```

OUTPUT:

```
Enter your username: sagar@1522
Enter your IP address: 192.168.1.10
```